

Intermediate Java

FRC Team 11025 - Garage Gang
Written by Tim Ellis



Prerequisites

- Classes
- Objects
- Methods
- Constructors
- Instance fields
- If statements
- Enhanced for loops
- Operators
- Comparisons

Covered Topics

- Inheritance
 - Interfaces
 - Enums
 - Switch Cases
 - Threads and Suppliers
 - Math library
- 

Inheritance

- Imagine an animal class
- Instance fields
 - Name
 - Weight
- Methods
 - constructor
 - getName
 - getWeight
 - eat

```
public class Animal{  
    private String name;  
    private double weightIbs;  
  
    public Animal(String name, double weightIbs){  
        this.name = name;  
        this.weightIbs = weightIbs;  
    }  
  
    public String getName(){  
        return name;  
    }  
  
    public double getWeight(){  
        return weightIbs;  
    }  
  
    public void eat(){  
        System.out.println(getName() + ": nom nom nom gulp");  
    }  
}
```

Inheritance

- Now let's say we want to make a Dog class
- This means we have to rewrite the instance fields, constructor, and methods that animals have
- Unless we use inheritance
- Inheritance allows us to reuse functionality from other classes
- To inherit from another class, use the 'extends' keyword in the header

```
public class Dog extends Animal{
```

Inheritance

- Additionally, in the constructor we must call the constructor of the Animal class
- To reference that constructor we use `super(params)`
- This way our dog can have a name and weight

```
public Dog(String name, double weight, String favoriteToy) {  
    super(name, weight);  
    this.favoriteToy = favoriteToy;  
}
```

Inheritance

- Now that we are extending the functionality of the Animal class, we can add dog specific things like a favorite toy and a woof function

```
public class Dog extends Animal{

    private String favoriteToy;

    public Dog(String name, double weight, String favoriteToy) {
        super(name, weight);
        this.favoriteToy = favoriteToy;
    }

    public String getFavoriteToy(){
        return favoriteToy;
    }

    public void woof(){
        System.out.println(getName() + ": woof");
    }
}
```

Inheritance

- We can do the same for a Cat class

```
public class Cat extends Animal{

    private String nappingSpot;

    public Cat(String name, double weight, String nappingSpot) {
        super(name, weight);
        this.nappingSpot = nappingSpot;
    }

    public String getNappingSpot(){
        return nappingSpot;
    }

    public void meow(){
        System.out.println(getName() + ": meow");
    }
}
```

Inheritance

- Note: we actually consider objects that are Dogs and Cats to 'be' Animals because they extend from that class
- All objects secretly extend from the Object class
- This is why they are objects

```
/**
 * Class {@code Object} is the root of the class hierarchy.
 * Every class has {@code Object} as a superclass. All objects,
 * including arrays, implement the methods of this class.
 *
 * @see    java.lang.Class
 * @since  1.0
 */
public class Object {
```

Inheritance

- Now we can call 'eat' on an animal object, dog object, and cat object

```
Animal biscuit = new Animal(name: "Biscuit", weightIbs: 50);  
Dog milo = new Dog(name: "Milo", weight: 30, favoriteToy: "Frisbee");  
Cat buttons = new Cat(name: "Buttons", weight: 40, nappingSpot: "Couch");  
  
biscuit.eat();  
milo.eat();  
buttons.eat();
```

```
Biscuit: nom nom nom gulp  
Milo: nom nom nom gulp  
Buttons: nom nom nom gulp
```

Inheritance

- Additionally we can call Dog and Cat specific methods on dogs and cats

```
miло.woof();  
buttons.meow();  
  
System.out.println("Milo's favorite toy: " + miло.getFavoriteToy());  
System.out.println("Button's napping spot: " + buttons.getNappingSpot());
```

```
Milo: woof  
Buttons: meow  
Milo's favorite toy: Frisbee  
Button's napping spot: Couch
```

Inheritance

- Because they are all Animals, they can go into an array of animals

```
Animal[] animals = {biscuit, milo, buttons};

for(Animal animal : animals){
    System.out.println(animal.getName() + "'s weight is " + animal.getWeight());
}
```

```
Biscuit's weight is 50.0
Milo's weight is 30.0
Buttons's weight is 40.0
```

Inheritance

- Lets add a new method to Animal

```
public void jump(){  
    System.out.println(getName() + " jumped 1 ft");  
}
```

- Jumping is different for Dogs and Cats (cats can jump higher)
- Lets add a jump method to both

Dog.java

```
public void jump(){  
    System.out.println(getName() + " jumped 2 ft");  
}
```

Cat.java

```
public void jump(){  
    System.out.println(getName() + " jumped 5 ft");  
}
```

Inheritance

- Now if there is a method in a child class (Cat and Dog) that is the same name as the parent class (Animal), the method in the child class will take precedence

```
Animal[] animals = {biscuit, milo, buttons};  
  
for(Animal animal : animals){  
    System.out.println(animal.getName() + "'s weight is " + animal.getWeight());  
    animal.jump();  
}
```

```
Biscuit's weight is 50.0  
Biscuit jumped 1 ft  
Milo's weight is 30.0  
Milo jumped 2 ft  
Buttons's weight is 40.0  
Buttons jumped 5 ft
```

Inheritance

- FRC example
- Say you want to write code for an intake and a shooter
- They both need a 'periodic' method that is run over and over again to manage subsystem operation
- Sure enough, they both extend a SubsystemBase class that has this method

```
public class Shooter extends SubsystemBase {
```



Interfaces

- Imagine an xbox remote
 - What does the 'A' button do in general?
 - It depends
 - If you're playing minecraft, it might be the jump button
 - If you're controlling a robot, it might be intake
 - Same button different action
- 

Interfaces

- Let's say we're writing the code for a vending machine
- A vending machine is just a bunch of buttons. We'll represent them with methods. 3 for simplicity.

```
public interface VendingMachine {  
    public default void button1() {}  
    public default void button2() {}  
    public default void button3() {}  
}
```



Interfaces

- Right now we are only concerned with the buttons, not what the buttons do
- We use the keyword 'interface' so that we can create methods that don't have logic behind them yet
- We also use the word default

```
public interface VendingMachine {  
    public default void button1() {}  
    public default void button2() {}  
    public default void button3() {}  
}
```

Interfaces

- Now lets add the logic behind these buttons
- To do this we create a class and use the keyword 'implements' to implement the methods in VendingMachine
- We'll do Coca Cola, Lemonade, and Pepsi

```
public interface VendingMachine {  
    public default void button1() {}  
    public default void button2() {}  
    public default void button3() {}  
}
```

```
public class VendingMachineA implements VendingMachine{  
    public void button1() {  
        System.out.println(x: "Dispensing Coca Cola");  
    }  
    public void button2() {  
        System.out.println(x: "Dispensing Lemonade");  
    }  
    public void button3() {  
        System.out.println(x: "Dispensing Pepsi");  
    }  
}
```

Interfaces

- Uh oh, there's a new model of Vending machine now
- There are now two models of vending machines out there and we need code that works for both of them
- This new vending machine has
 - Cranberry Punch Spindrift
 - Ginger Lime Spindrift
 - Cherry Spindrift

```
public interface VendingMachine {  
    public default void button1() {}  
    public default void button2() {}  
    public default void button3() {}  
}
```

```
public class VendingMachineB implements VendingMachine{  
    public void button1() {  
        System.out.println(x: "Dispensing Cranberry Punch Spindrift");  
    }  
    public void button2() {  
        System.out.println(x: "Dispensing Ginger Lime Spindrift");  
    }  
    public void button3() {  
        System.out.println(x: "Dispensing Cherry Spindrift");  
    }  
}
```

Interfaces

- Now we can control which model the code works for using a single boolean
- Notice how the 'runVendingMachine' method uses the same button methods but they have different logic behind them depending on the type of machine

```
public static void main(String[] args) throws Exception {  
    boolean useNewVendingMachine = false;  
    VendingMachine machine;  
    if(useNewVendingMachine){  
        machine = new VendingMachineB();  
    } else {  
        machine = new VendingMachineA();  
    }  
    runVendingMachine(machine);  
}  
  
public static void runVendingMachine(VendingMachine machine){  
    machine.button1();  
    machine.button2();  
    machine.button3();  
}
```

```
boolean useNewVendingMachine = false;
```

```
Dispensing Coca Cola  
Dispensing Lemonade  
Dispensing Pepsi
```

```
boolean useNewVendingMachine = true;
```

```
Dispensing Cranberry Punch Spindrift  
Dispensing Ginger Lime Spindrift  
Dispensing Cherry Spindrift
```

Interfaces

- FRC Example:
- Let's say we want to write code to control a real intake and a simulated intake
- Both will need an 'intake' method
- The intake button on the controller should work regardless of if it's real or simulated

```
public interface IntakeIO {
```

```
    public default void setFrontVel(AngularVelocity angle) {}
```

```
public class IntakeIOReal implements IntakeIO {
```

```
    private final SparkFlex frontRollers =  
        new SparkFlex(Constants.CanIDs.INTAKE_FRONT_ROLLERS_CAN_ID, MotorType.kBrushless);
```

```
@Override  
public void setFrontVel(AngularVelocity vel) {  
    frontRollers  
        .getClosedLoopController()  
        .setReference(  
            vel.in(Units.RPM) / IntakeConstants.FrontRollers.GEAR_RATIO,  
            ControlType.kMAXMotionVelocityControl);  
}
```

```
public class IntakeIOSim implements IntakeIO {
```

```
    private final GenericMotorSim frontRoller =  
        new GenericMotorSim(  
            DCMotor.getNEO(numMotors: 1),  
            IntakeConstants.FrontRollers.GEAR_RATIO,  
            IntakeConstants.FrontRollers.MOI);
```

```
@Override  
public void setFrontVel(AngularVelocity vel) {  
    frontRoller.setVelocity(vel.div(IntakeConstants.FrontRollers.GEAR_RATIO));  
}
```

Enums and Switch Cases

- Enum is short for enumeration
- Let's say you wanted to create a method called getFavoriteDrink
- getFavoriteDrink would require a parameter, the person
- Heres an example implementation:

```
public static void main(String[] args) throws Exception {  
    System.out.println(getFavoriteDrink(name: "Tim"));  
    System.out.println(getFavoriteDrink(name: "Nat"));  
    System.out.println(getFavoriteDrink(name: "Drew"));  
    System.out.println(getFavoriteDrink(name: "Mandy"));  
}  
  
public static String getFavoriteDrink(String name){  
    if(name == "Tim"){  
        return "Lemonade";  
    } else if (name == "Nat"){  
        return "Red Wine";  
    } else if (name == "Drew"){  
        return "Gatorade";  
    } else {  
        return "Water";  
    }  
}
```

Lemonade
Red Wine
Gatorade
Water

Enums and Switch Cases

- There's a cleaner way to do this. We can create an enum for name and an enum for drink. These will store the available names and types of drinks.
- Now it's clear what names and drinks are available
- There are a finite number of names and a finite number of drinks (with Strings, there could be infinite)

```
public enum Name{
    Tim,
    Nat,
    Drew,
    Mandy
}

public enum Drink{
    Lemonade,
    Red_Wine,
    Gatorade,
    Water
}

Run | Debug
public static void main(String[] args) throws Exception {
    System.out.println(getFavoriteDrink(Name.Tim));
    System.out.println(getFavoriteDrink(Name.Nat));
    System.out.println(getFavoriteDrink(Name.Drew));
    System.out.println(getFavoriteDrink(Name.Mandy));
}

public static Drink getFavoriteDrink(Name name){
    if(name == Name.Tim){
        return Drink.Lemonade;
    } else if (name == Name.Nat){
        return Drink.Red_Wine;
    } else if (name == Name.Drew){
        return Drink.Gatorade;
    } else {
        return Drink.Water;
    }
}
```

Enums and Switch Cases

- We can make this even cleaner by replacing the if statements with a switch case
- Ideally the switch case covers every possible name. We can add a 'default' case for if none of the other cases match name (ex: if name is Mandy).

```
public static Drink getFavoriteDrink(Name name){  
    Drink drink;  
    switch (name) {  
        case Tim:  
            drink = Drink.Lemonade;  
            break;  
        case Nat:  
            drink = Drink.Red_Wine;  
            break;  
        case Drew:  
            drink = Drink.Gatorade;  
            break;  
        default:  
            drink = Drink.Water;  
            break;  
    }  
    return drink;  
}
```

Threads and Suppliers

- Imagine we make a clock object.
- We want this clock object to keep running even when other code is being executed
- This means we need code to happen in parallel
- For this, we'll need different threads
- We can create a thread object, give it a method to run, and start the thread
- Instead of counting seconds, we'll count loops of the while loop

```
public class Clock {  
    private int ticks = 0;  
    public Clock(){  
        Thread ticker = new Thread(() -> tickLoop());  
        ticker.start();  
    }  
  
    private void tickLoop(){  
        while(ticks < 8000){  
            ticks++;  
        }  
    }  
  
    public int getTicks(){  
        return ticks;  
    }  
}
```



Threads and Suppliers

- Note that when we introduce new threads, we make our program non-deterministic. This means it can act differently from run to run depending on the state of the computer hardware.





Threads

- Now in our main function, lets store the time in a variable and print it repeatedly
- Uh oh, it's 0 over and over again
- This is because we stored 0 to the time variable and never updated that

```
public static void main(String[] args) throws Exception {
    Clock clock = new Clock();
    int time = clock.getTicks();
    while(time < 8000){
        System.out.println(time);
    }
}
```

[illegible]

Threads and Suppliers

- One solution would be to update time from the clock every loop
- Note that we aren't printing every tick because printing takes time to happen, so the clock ticks a lot in between prints

```
public static void main(String[] args) throws Exception {  
    Clock clock = new Clock();  
    int time = clock.getTicks();  
    while(time < 60000){  
        time = clock.getTicks();  
        System.out.println(time);  
    }  
}
```

```
0  
23502  
32505  
37174  
42994  
50453  
57854  
60000
```

Threads and Suppliers

- Another solution would be to use a supplier
- A supplier is basically an arrow pointing to where to get information rather than that information itself
- The syntax is `Supplier<Object you want to get from the supplier>`
- Note Suppliers supply objects, not primitives. We need to use the Integer object rather than the int primitive.

```
public static void main(String[] args) throws Exception {  
    Clock clock = new Clock();  
    Supplier<Integer> time = clock::getTicks;  
    while(time.get() < 6000000){  
        System.out.println(time.get());  
    }  
}
```

```
340741  
465708  
489815  
502783  
514748  
534532  
559749  
560127  
560127  
560243
```

Math Library

- If you need to do non-basic math in Java, use Math.[equation]
- You can also get constants like Math.PI and Math.E

```
System.out.println(Math.min(a: 5, b: 10));  
System.out.println(Math.max(a: 5, b: 10));  
System.out.println(Math.clamp(value: 2, min: 5, max: 10));  
System.out.println(Math.abs(-6));  
System.out.println(Math.sqrt(a: 25));  
System.out.println(Math.pow(a: 5, b: 10));  
System.out.println(Math.sin(Math.PI/2));  
System.out.println(Math.atan2(y: 0, -1));  
System.out.println(Math.E);  
// and much much more
```

```
5  
10  
5  
6  
5.0  
9765625.0  
1.0  
3.141592653589793  
2.718281828459045
```

Math Library

- Challenge: reproduce this in Java

$$\frac{(\pi \cdot \log(5))}{5^3 - \text{abs}(-9)} - \sin(4) \cdot \tan(e)$$

$$= -0.32204702$$